

OOPS WITH C++

Unit 1 + Unit 2 + Unit 3 — Final Exam Notes

Easy English • Full Theory • Turbo C++ Programs • Outputs • Emojis

■ UNIT 1 — CONCEPTS OF OOPS

■ 1.1 Procedural Programming vs OOPS

Procedural Programming: Procedural programming is a programming approach in which a program is divided into functions or procedures.

Object Oriented Programming: OOPS is a programming approach in which a program is organized using classes and objects.

Procedural Programming	OOPS
Focuses on functions	Focuses on objects
Data and functions are separate	Data and functions are combined
Top-to-bottom approach	Object-oriented approach
Less data security	Better data security
Less reusable	More reusable
Example: C	Example: C++

■ **Remember:** Procedural → Functions OOPS → Classes + Objects

■ 1.2 C++ Library / Header Files

Header files contain predefined functions and facilities used in C++.

Header File	Use
iostream.h	cin, cout
conio.h	clrscr(), getch()
string.h	String functions
math.h	Mathematical functions
stdio.h	Input/Output

```
#include<iostream.h>
#include<conio.h>

void main()
{
    clrscr();

    cout<<"Welcome to C++";

    getch();
}
```

■■ Output

Welcome to C++

■ 1.3 Data Types in C++

A data type specifies the type of value stored in a variable.

- int → Integer
- float → Decimal value
- double → Large decimal value
- char → Single character
- void → No value

```
#include<iostream.h>
#include<conio.h>

void main()
{
    int age=19;
    float marks=85.5;
    char grade='A';

    clrscr();

    cout<<"Age = "<<age;
    cout<<"\nMarks = "<<marks;
    cout<<"\nGrade = "<<grade;

    getch();
}
```

■ Output

```
Age = 19
Marks = 85.5
Grade = A
```

■ 1.4 STRING

A string is a collection of characters ending with a null character \0.

```
H E L L O \0
```

■ 1.4.1 Character Array

A character array is used to store a string.

```
#include<iostream.h>
#include<conio.h>

void main()
{
    char name[20]="Pritesh";

    clrscr();

    cout<<"Name = "<<name;

    getch();
}
```

■ Output

```
Name = Pritesh
```

■ 1.4.2 Pointer to Character Array

A character pointer can point to a character/string.

```
#include<iostream.h>
#include<conio.h>

void main()
{
    char *name="Pritesh";

    clrscr();

    cout<<"Name = "<<name;

    getch();
}
```

■■ Output

Name = Pritesh

■ 1.4.3 string.h Functions

string.h provides functions for string manipulation.

- strlen() → Find length
- strcpy() → Copy string
- strcat() → Join strings
- strcmp() → Compare strings
- strrev() → Reverse string

■ strlen()

Finds the length of a string.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>

void main()
{
    char name[]="Pritesh";

    clrscr();

    cout<<"Length = "<<strlen(name);

    getch();
}
```

■■ Output

Length = 7

■ strcpy()

Copies one string into another.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>

void main()
{
    char a[20]="Pritesh";
    char b[20];

    clrscr();

    strcpy(b,a);

    cout<<"Copied String = "<<b;

    getch();
}
```

■■ Output

Copied String = Pritesh

■ strcat()

Joins two strings.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>

void main()
{
    char a[30]="Hello ";
    char b[]="Pritesh";

    clrscr();

    strcat(a,b);

    cout<<a;

    getch();
}
```

■■ Output

Hello Pritesh

■ strcmp()

Compares two strings.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>

void main()
{
    char a[]="ABC";
    char b[]="ABC";

    clrscr();

    if(strcmp(a,b)==0)
        cout<<"Strings are Equal";
    else
        cout<<"Strings are Not Equal";

    getch();
}
```

■■ Output

```
Strings are Equal
```

■ strrev()

Reverses a string.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>

void main()
{
    char name[]="Pritesh";

    clrscr();

    cout<<"Reverse = "<<strrev(name);

    getch();
}
```

■■ Output

```
Reverse = hsetirP
```

■ 1.5 CLASS AND OBJECT

Class: A class is a blueprint/template for creating objects.

A class contains data members and member functions.

Object: An object is an instance of a class.

Example: **Student s1;** — Student = Class, s1 = Object.

```
#include<iostream.h>
#include<conio.h>

class Student
{
    public:
        int roll;

        void display()
        {
            cout<<"Roll No = "<<roll;
        }
};

void main()
{
    Student s1;

    clrscr();

    s1.roll=101;
    s1.display();

    getch();
}
```

■■ Output

```
Roll No = 101
```

■ UNIT 2 — DATA ENCAPSULATION AND INHERITANCE

■ 2.1 ACCESS CONTROLS

C++ has three access specifiers: public, private and protected.

Public: Accessible from outside the class.

Private: Accessible only inside the same class. By default, class members are private.

Protected: Accessible inside the class and derived class.

Public	Private	Protected
Accessible from anywhere	Same class only	Same class + derived class
Less restriction	Highest restriction	Medium restriction

■ 2.2 SIMPLE CLASS

A class contains member variables and member functions.

Member Variable: A variable declared inside a class.

Member Function: A function declared inside a class.

```
#include<iostream.h>
#include<conio.h>

class Student
{
    public:
        int roll;

        void display()
        {
            cout<<"Roll No = "<<roll;
        }
};

void main()
{
    Student s;

    clrscr();

    s.roll=101;
    s.display();

    getch();
}
```

■■ Output

```
Roll No = 101
```

■ 2.3 ENUM

enum means enumeration. It is a user-defined data type containing named integer constants.

```
enum Day
{
    Monday,
    Tuesday,
    Wednesday
};
```

By default: Monday = 0, Tuesday = 1, Wednesday = 2.

```
#include<iostream.h>
#include<conio.h>

enum Day
{
    Monday,
    Tuesday,
    Wednesday,
    Thursday,
    Friday
};

void main()
{
    Day d;

    clrscr();

    d=Wednesday;

    cout<<"Day Value = "<<d;

    getch();
}
```

■■ Output

Day Value = 2

■ 2.4 DATA HIDING

Data hiding means restricting direct access to class data. Private members are commonly used for data hiding.

```
#include<iostream.h>
#include<conio.h>

class Account
{
    private:
        int balance;

    public:
        void setBalance(int b)
        {
            balance=b;
        }

        void showBalance()
        {
            cout<<"Balance = "<<balance;
        }
};

void main()
{
    Account a;

    clrscr();

    a.setBalance(5000);
    a.showBalance();

    getch();
}
```

■■ Output

Balance = 5000

■ ABSTRACTION

Abstraction means showing necessary information and hiding internal implementation details.

■ Example: ATM — Enter PIN, select withdrawal, enter amount and receive money. The internal processing is hidden.

```
#include<iostream.h>
#include<conio.h>

class ATM
{
    private:
        int balance;

    public:
        void setBalance(int b)
        {
            balance=b;
        }

        void withdraw(int amount)
        {
            if(amount<=balance)
                cout<<"Withdrawal Successful";
            else
                cout<<"Insufficient Balance";
        }
};

void main()
{
    ATM a;

    clrscr();

    a.setBalance(5000);
    a.withdraw(2000);

    getch();
}
```

■ Output

Withdrawal Successful

■ ENCAPSULATION

Encapsulation means wrapping data and functions together inside one class.

```
Class
    ■■■ Data
    ■■■ Functions
```



```

#include<iostream.h>
#include<conio.h>

class Student
{
    private:
        int marks;

    public:
        void setMarks(int m)
        {
            marks=m;
        }

        void display()
        {
            cout<<"Marks = "<<marks;
        }
};

void main()
{
    Student s;

    clrscr();

    s.setMarks(85);
    s.display();

    getch();
}

```

■■ Output

Marks = 85

Data Hiding	Abstraction	Encapsulation
Restricts data access	Hides implementation	Wraps data + functions
Uses access control	Shows essential information	Uses class

■ 2.5 INHERITANCE

Inheritance is an OOPS feature in which a child/derived class acquires properties and functions of a parent/base class.

- ■■ Code reusability
- Reduces duplicate code
- Easy maintenance
- Creates parent-child relationship

Types: Single, Multiple, Multilevel, Hierarchical, Hybrid

■ 1. SINGLE INHERITANCE

```
#include<iostream.h>
#include<conio.h>

class Father
{
    public:
        void property()
        {
            cout<<"Father Property";
        }
};

class Son : public Father
{
    public:
        void show()
        {
            cout<<"\nSon";
        }
};

void main()
{
    Son s;

    clrscr();

    s.property();
    s.show();

    getch();
}
```

■■ Output

```
Father Property
Son
```

■ 2. MULTIPLE INHERITANCE

```

#include<iostream.h>
#include<conio.h>

class Father
{
    public:
        void fatherProperty()
        {
            cout<<"Father Property\n";
        }
};

class Mother
{
    public:
        void motherProperty()
        {
            cout<<"Mother Property\n";
        }
};

class Son : public Father, public Mother
{
    public:
        void sonProperty()
        {
            cout<<"Son Property";
        }
};

void main()
{
    Son s;

    clrscr();

    s.fatherProperty();
    s.motherProperty();
    s.sonProperty();

    getch();
}

```

■■ Output

```

Father Property
Mother Property
Son Property

```

■ 3. MULTILEVEL INHERITANCE

```

#include<iostream.h>
#include<conio.h>

class Grandfather
{
    public:
        void grandProperty()
        {
            cout<<"Grandfather Property\n";
        }
};

class Father : public Grandfather
{
    public:
        void fatherProperty()
        {
            cout<<"Father Property\n";
        }
};

class Son : public Father
{
    public:
        void sonProperty()
        {
            cout<<"Son Property";
        }
};

void main()
{
    Son s;

    clrscr();

    s.grandProperty();
    s.fatherProperty();
    s.sonProperty();

    getch();
}

```

■■ Output

```

Grandfather Property
Father Property
Son Property

```

■ 4. HIERARCHICAL INHERITANCE

```

#include<iostream.h>
#include<conio.h>

class Father
{
    public:
        void property()
        {
            cout<<"Father Property\n";
        }
};

class Son : public Father
{
    public:
        void son()
        {
            cout<<"Son";
        }
};

class Daughter : public Father
{
    public:
        void daughter()
        {
            cout<<"Daughter";
        }
};

void main()
{
    Son s;
    Daughter d;

    clrscr();

    s.property();
    s.son();

    cout<<"\n";

    d.property();
    d.daughter();

    getch();
}

```

■■ Output

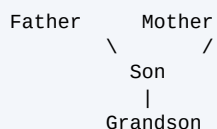
```

Father Property
Son
Father Property
Daughter

```

■ 5. HYBRID INHERITANCE

Hybrid inheritance is a combination of two or more types of inheritance.



■ 2.6 CONSTRUCTOR

A constructor is a special member function used to initialize an object.

- Same name as class
- No return type

- Automatically called
- Used for initialization
- Can be overloaded

■ DEFAULT CONSTRUCTOR

A constructor with no arguments.

```
#include<iostream.h>
#include<conio.h>

class Student
{
    public:
        Student()
        {
            cout<<"Default Constructor";
        }
};

void main()
{
    Student s;

    clrscr();

    getch();
}
```

■■ Output

Default Constructor

■ PARAMETERIZED CONSTRUCTOR

A constructor that accepts parameters.

```
#include<iostream.h>
#include<conio.h>

class Student
{
    int roll;

    public:
        Student(int r)
        {
            roll=r;
        }

        void display()
        {
            cout<<"Roll No = "<<roll;
        }
};

void main()
{
    Student s(101);

    clrscr();

    s.display();

    getch();
}
```

■■ Output

Roll No = 101

Default	Parameterized
No arguments	Has arguments
Student()	Student(int r)
Default initialization	Specific initialization

■ COPY CONSTRUCTOR

A copy constructor creates a new object by copying an existing object of the same class.

```
ClassName(ClassName &object)
{
}
```

```
#include<iostream.h>
#include<conio.h>

class Student
{
    int roll;

    public:
        Student(int r)
        {
            roll=r;
        }

        Student(Student &s)
        {
            roll=s.roll;
        }

        void display()
        {
            cout<<"Roll No = "<<roll;
        }
};

void main()
{
    Student s1(101);
    Student s2(s1);

    clrscr();

    cout<<"Object 1: ";
    s1.display();

    cout<<"\nObject 2: ";
    s2.display();

    getch();
}
```

■■ Output

```
Object 1: Roll No = 101
Object 2: Roll No = 101
```

■ DESTRUCTOR

A destructor is a special member function that is automatically called when an object is destroyed.

- Same name as class
- Starts with ~
- No return type

- No arguments
- Cannot be overloaded
- Automatically called

```
#include<iostream.h>
#include<conio.h>

class Demo
{
    public:
        Demo()
        {
            cout<<"Constructor Called\n";
        }

        ~Demo()
        {
            cout<<"Destructor Called";
        }
};

void main()
{
    clrscr();

    Demo d;

    getch();
}
```

■■ Output

```
Constructor Called
Destructor Called
```

■ UNIT 3 — POLYMORPHISM

■ 3.1 POLYMORPHISM

Polymorphism means “**many forms.**” In OOPS, the same name can have different forms or behaviors.

```
add(int,int)
add(int,int,int)
```

■ 3.2 TYPES OF POLYMORPHISM

1■■ Compile-Time Polymorphism: Decision is made during compilation. Examples: Function Overloading, Operator Overloading.

2■■ Run-Time Polymorphism: Decision is made during execution. Examples: Function Overriding, Virtual Function.

Compile-Time	Run-Time
Decision at compile time	Decision at runtime
Early binding	Late binding
Function overloading	Function overriding
Operator overloading	Virtual function

■ 3.3 FUNCTION OVERLOADING

Function overloading means multiple functions have the same name but different parameters. Parameters can differ by number, type or order.


```

#include<iostream.h>
#include<conio.h>

class Calculator
{
    public:

        void add(int a,int b)
        {
            cout<<"Sum = "<<a+b;
        }

        void add(int a,int b,int c)
        {
            cout<<"\nSum = "<<a+b+c;
        }
};

void main()
{
    Calculator c;

    clrscr();

    c.add(10,20);
    c.add(10,20,30);

    getch();
}

```

■■ Output

```

Sum = 30
Sum = 60

```

■ FUNCTION OVERRIDING

Function overriding occurs when a derived class provides its own implementation of a function already present in the base class.

```

#include<iostream.h>
#include<conio.h>

class Father
{
    public:
        void show()
        {
            cout<<"Father's Function";
        }
};

class Son : public Father
{
    public:
        void show()
        {
            cout<<"Son's Function";
        }
};

void main()
{
    Son s;

    clrscr();

    s.show();

    getch();
}

```

■■ Output

■ OVERLOADING vs OVERRIDING

Overloading	Overriding
Same function name	Same function name
Different parameters	Derived class gives new implementation
Usually same class	Base + derived class
Compile-time	Run-time with virtual function
Early binding	Late binding

■ FRIEND FUNCTION

A friend function is not a member of the class, but it can access private and protected members. It is declared using friend.

```
#include<iostream.h>
#include<conio.h>

class Student
{
    private:
        int marks;

    public:
        Student()
        {
            marks=85;
        }

        friend void display(Student);
};

void display(Student s)
{
    cout<<"Marks = "<<s.marks;
}

void main()
{
    Student s;

    clrscr();

    display(s);

    getch();
}
```

■■ Output

Marks = 85

■ VIRTUAL FUNCTION

A virtual function is a member function declared using the virtual keyword in the base class. It is used for run-time polymorphism.

```

#include<iostream.h>
#include<conio.h>

class Father
{
    public:
        virtual void show()
        {
            cout<<"Father";
        }
};

class Son : public Father
{
    public:
        void show()
        {
            cout<<"Son";
        }
};

void main()
{
    Father *ptr;
    Son s;

    clrscr();

    ptr=&s;
    ptr->show();

    getch();
}

```

■■ Output

Son

■ PURE VIRTUAL FUNCTION

A pure virtual function is a virtual function with no implementation in the base class. It is declared using virtual void show() = 0.

A class containing a pure virtual function is called an abstract class.

```

#include<iostream.h>
#include<conio.h>

class Animal
{
    public:
        virtual void sound()=0;
};

class Dog : public Animal
{
    public:
        void sound()
        {
            cout<<"Dog Barks";
        }
};

void main()
{
    Dog d;

    clrscr();

    d.sound();

    getch();
}

```

■■ Output

Dog Barks

■ OPERATOR OVERLOADING

Operator overloading means giving a special meaning to an operator for objects.

Examples: +, -, *, /, ==

```
#include<iostream.h>
#include<conio.h>

class Number
{
    int x;

public:
    Number()
    {
        x=0;
    }

    Number(int a)
    {
        x=a;
    }

    Number operator +(Number n)
    {
        Number temp;
        temp.x=x+n.x;
        return temp;
    }

    void display()
    {
        cout<<"Value = "<<x;
    }
};

void main()
{
    Number n1(10),n2(20),n3;

    clrscr();

    n3=n1+n2;

    n3.display();

    getch();
}
```

■■ Output

Value = 30

■ ■ FINAL GOLDEN MEMORY LINE

■ Class → Object

■ Data → Hiding

■ Data + Function → Encapsulation

■ Hide Details → Abstraction

■ Parent → Child → Inheritance

- Many Forms → Polymorphism
- Same Name + Different Parameters → Overloading
- Child Changes Parent Function → Overriding
- Friend → Private Access
- Virtual → Run Time
- = 0 → Pure Virtual
- Operator + Object → Operator Overloading

■ UNIT 1 + 2 + 3 — MOST IMPORTANT

■ **Unit 1:** Procedural vs OOPS • Header Files • Data Types • Character Array • Character Pointer • strlen() • strcpy() • strcat() • strcmp() • strrev() • Class & Object

■ **Unit 2:** Public / Private / Protected • Member Variables / Functions • enum • Data Hiding • Abstraction • Encapsulation • Inheritance • 5 Types of Inheritance • Constructor • Default Constructor • Parameterized Constructor • Copy Constructor • Destructor

■ **Unit 3:** Polymorphism • Compile-Time / Run-Time • Function Overloading • Function Overriding • Overloading vs Overriding • Friend Function • Virtual Function • Pure Virtual Function • Operator Overloading